TECHNICAL REPORT: DISTANCE VECTOR (DV) VS LINK STATE (LS) ROUTING PROTOCOLS

An Investigation of Update Overhead, Convergence Rounds, and Route Recovery Under Link Failure

Student Name: Prachi Modi

Student ID: 25CS045

Course: Fundamentals of Computer Networks (ITUC201)

Department: Department of Computer Science & Engineering

Institution: CSPIT, Charusat University

Division: 1

1. Objectives

In computer network architectures, routing protocols determine the optimal path that data packets take from a source node to a destination node across interconnected routers. The two most fundamental paradigms taught in routing theory are **Distance Vector (DV)** protocols (such as RIP, based on the distributed Bellman-Ford algorithm) and **Link State (LS)** protocols (such as OSPF and IS-IS, based on Dijkstra's Shortest Path First algorithm).

While textbook theory clearly explains the conceptual differences, it can be difficult to appreciate how these differences affect simulated control-message overhead and convergence behavior during network topology failures without implementing and measuring the algorithms directly.

The primary objectives of this project are:

- **Develop an Educational Simulation Engine in Python:** Implement synchronous, discrete-round simulation models for both Distance Vector and Link State routing algorithms from the ground up without relying on black-box routing libraries.
- **Standardize a Common Five-Node Topology:** Build a matched 5-node test network (`A` , `B` , `C` , `D` , `E`) with 7 bidirectional links of varied link costs to serve as a fair, identical testing ground for both algorithms.
- **Simulate Network Failure and Recovery:** Inject a controlled link failure by removing the link between Router `B` and Router `D` (link cost = 2) and evaluate how each protocol adapts.
- **Quantitatively Measure Update Overhead and Convergence Rounds:** Record the exact number of control message transmission attempts and the number of discrete synchronous rounds required to reach a stable state across three test scenarios:
  - *Baseline Scenario:* Normal, healthy network starting from cold boot.
  - *Fresh Start After Failure Scenario:* Network starting cold without link B–D present.
  - *Dynamic Recovery Scenario:* Network transitioning directly from the converged baseline state to a recovered state after link B–D is severed mid-operation.
- **Implement Automated Route Verification:** Automatically validate that the computed shortest-path costs and next-hop forwarding decisions match mathematical ground truth calculated independently using NetworkX's Dijkstra implementation.

- **Analyze Engineering Trade-offs:** Evaluate the trade-off between initial setup message overhead and dynamic adaptation efficiency during link failure recovery.

2. Methodology & Experimental Design

2.1 Theoretical Framework

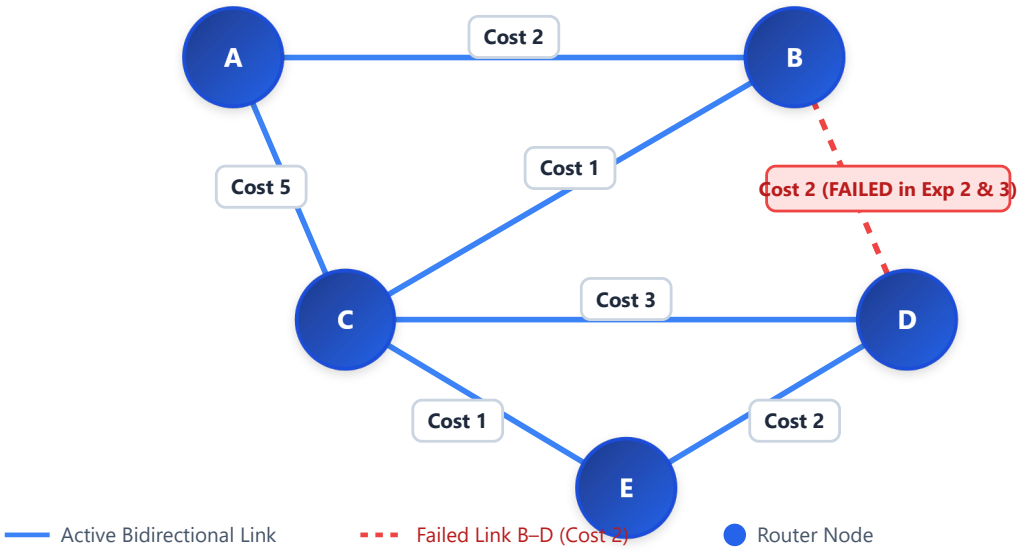
Distance Vector (DV) Mechanism: Distance Vector routing relies on distributed computation. Each router maintains a routing table containing its current estimated distance to every possible destination and the corresponding next-hop neighbor. Routers do not know the complete map of the network; they only know their own direct links and the distance vectors advertised by their immediate neighbors. In each round, every router shares its distance vector with its direct neighbors and updates its table using the Bellman-Ford equation:

```
D_u(d) = min_{v ∈ Neighbors(u)} { c(u, v) + D_v(d) }
```

Link State (LS) Mechanism: Link State routing decouples topology dissemination from shortest-path computation. Each router packages its local link connectivity into a *Link State Advertisement (LSA)*, floods this advertisement to all routers across the network, builds an identical global *Link State Database (LSDB)*, and independently executes Dijkstra's Shortest Path First (SPF) algorithm locally to build its forwarding table.

2.2 Network Topology Specification

A 5-node topology with 7 bidirectional links (representing 14 directed links in normal operation) was configured in `network_config.py`. The layout and exact link costs are illustrated below:



Network Topology Diagram: 5 routers with 7 weighted links. In failure experiments, the B–D link (cost 2) is disabled.

2.3 Experimental Scenarios

1. **Baseline Scenario:** All 5 nodes and 7 links are active (14 directed links). Evaluates cold-start discovery and convergence.
2. **Fresh Start After Failure:** Initialized without link (B, D). 5 nodes and 6 links active (12 directed links). Cold start under degraded conditions.
3. **Dynamic Recovery Scenario:** Starts directly from the converged baseline state of Experiment 1. Link (B, D) is severed mid-operation. Evaluates adaptive failure recovery without resetting routing tables.

3. Implementation Details

The simulation was implemented in Python 3 across two modules: `network_config.py` (topology constants) and `main.py` (simulation logic, measurements, and verification).

3.1 Distance Vector Implementation

In `main.py`, Distance Vector executes in synchronous discrete rounds:

- **Initialization:** Each router initializes its distance to itself as 0, to direct neighbors as the direct link cost, and to all other nodes as infinity (`inf`).
- **Synchronous Relaxation:** In each round, routers evaluate incoming advertisements against their current distances. When a lower-cost path is discovered, distance estimates and next hops are updated, tripping the round change flag.
- **Loop Prevention in Dynamic Recovery:** When link (B, D) drops, naive Bellman-Ford could experience the count-to-infinity problem. To ensure clean convergence, **Split Horizon with Poisoned Reverse** was implemented in `run_dv_recovery`:

  ```
  advertised_cost = old_distances[neighbor][destination]
  if old_next_hops[neighbor][destination] == node:
      advertised_cost = inf
  ```

  If a router routes to destination `dest` via neighbor `N`, it advertises an infinite cost back to `N`, preventing circular dependency loops.

3.2 Link State Implementation

Link State executes in two distinct phases:

- **Flooding Phase:** Routers generate Link State Advertisements (LSAs) containing their direct neighbor costs. Flooding is managed via round-based frontier tracking. Routers forward new LSAs to all neighbors except the incoming sender (`if neighbour == incoming_neighbour: continue`). Each transmission attempt is recorded.
- **Local SPF Computation:** Once all LSAs are received and the flooding frontier is empty, each router independently executes Dijkstra's algorithm using Python's `heapq` priority queue on its local Link State Database (LSDB) to compute shortest paths and next hops.

- **Event-Driven Recovery:** In `run_ls_recovery`, only the incident routers directly connected to the severed link (Routers B and D) generate updated LSAs. Routers A, C, and E forward the modified LSAs and update their local databases without generating redundant advertisements.

3.3 Automated Independent Validation

To eliminate reporting errors, `main.py` implements automated validation against NetworkX's `single_source_dijkstra_path_length` and next-hop consistency checks (`validate_distances` and `validate_next_hops`). All six experimental configurations successfully passed both distance and next-hop validation.

4. Observations and Experimental Results

4.1 Experimental Measurements

The simulation was executed and all measurements were logged directly to `results/results.csv`:

Experiment Scenario	Protocol	Rounds	Message Count	Metric Counted	Validation Status
1. Baseline Network	Distance Vector (DV)	3	42	DV vector advertisement attempts	PASS
1. Baseline Network	Link State (LS)	3	50	LSA transmission attempts	PASS
2. Fresh Start After Failure	Distance Vector (DV)	3	36	DV vector advertisement attempts	PASS
2. Fresh Start After Failure	Link State (LS)	3	40	LSA transmission attempts	PASS
3. Recovery After Failure	Distance Vector (DV)	3	36	DV vector advertisement attempts	PASS
3. Recovery After Failure	Link State (LS)	3	16	LSA transmission attempts	PASS

4.2 Empirical Analysis and Generated Graphs

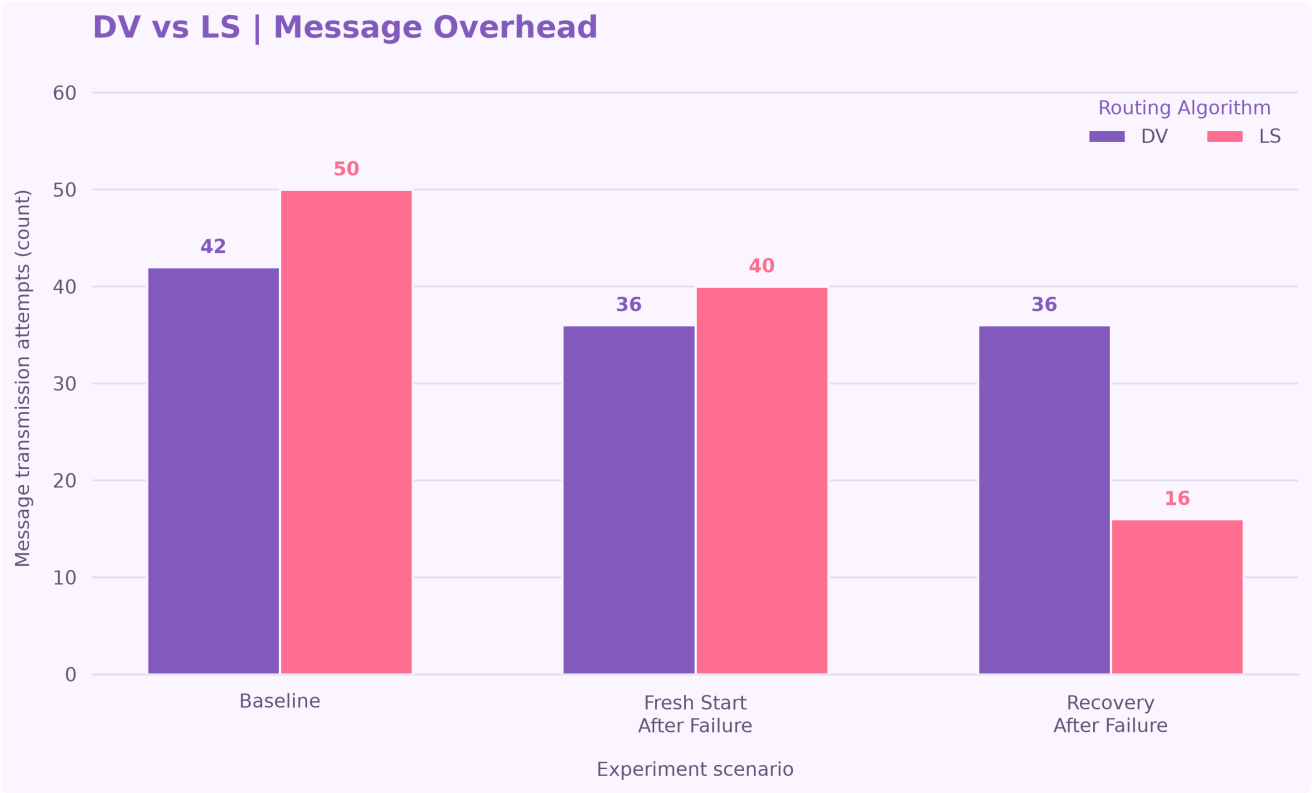


Figure 1: Control message transmission attempts for Distance Vector and Link State across all three experimental scenarios.

Interpretation: In the baseline startup experiment, Distance Vector recorded 42 message advertisement attempts while Link State recorded 50 LSA transmission attempts. The higher baseline overhead in Link State is caused by flooding duplication across mesh paths. In the dynamic recovery experiment, however, Link State required only **16 message transmission attempts** compared to **36 for Distance Vector** (a 55.6% reduction). Because LS updates are event-driven, only the affected nodes (B and D) generated changed LSAs, whereas DV required full periodic vector exchanges across all 12 directed links over 3 rounds to achieve and confirm stability.

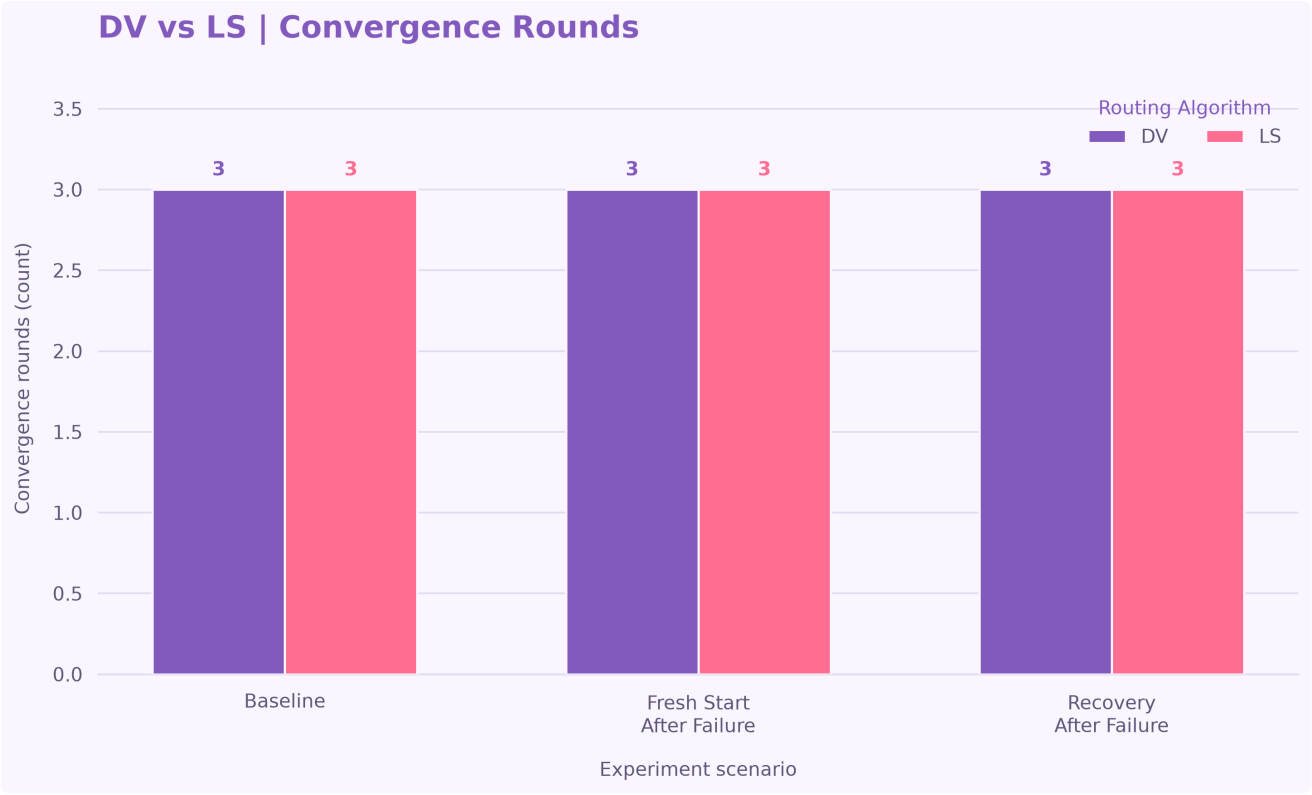


Figure 2: Discrete synchronous convergence rounds required by Distance Vector and Link State across all three experimental scenarios.

Interpretation: Both algorithms required exactly 3 synchronous rounds across all three scenarios. In DV, two rounds were necessary for distance updates to propagate across the network diameter, followed by one confirmation round with zero updates. In LS, three rounds were required for the LSA flooding wave to reach the empty frontier condition across the 5-node topology.

4.3 Verification of Technical Claims and Metric Definitions

- **Message Counts Represent Transmission Attempts, Not Bandwidth:** The reported message counts measure algorithm-specific control packet transmission attempts over directed links. They do not represent total byte volume or raw network bandwidth consumption. A Distance Vector advertisement contains the router's entire routing table (multiple destination-distance pairs), whereas an individual LSA contains only local adjacency information.
- **Round Counts Are Implementation-Specific:** Although both protocols report 3 rounds, they employ fundamentally different stopping conventions. DV rounds involve distributed message passing and table relaxation until a full round yields zero cost changes. LS rounds track the synchronous flooding wave until the frontier is exhausted, after which Dijkstra SPF is computed locally in memory. Consequently, identical round counts in this synchronous model do not prove identical real-world convergence speed or wall-clock elapsed time.
- **Rerouting Path Behavior:** Prior to failure, Router B reached Router D directly via link (B, D) at cost 2. Following the failure, traffic rerouted via Router C:

```
Path: B -> C -> D (Cost = 1 + 3 = 4)  or  Path: B -> C -> E -> D (Cost = 1 + 1 + 2 = 4)
```

Both implementations updated to cost 4 with next hop C, passing all NetworkX verification checks.

- **Loop Freedom Scope:** In our simulation, synchronous lockstep execution and Split Horizon with Poisoned Reverse prevented transient routing loops in DV. In real-world asynchronous networks, temporary loops can still occur during topology transitions in both protocol families before all databases or tables synchronize.

5. Conclusions & Key Takeaways

This investigation empirically demonstrates the operational trade-offs between Distance Vector and Link State routing algorithms:

1. **Initial Setup vs. Failure Adaptation:** Distance Vector has low implementation complexity and relatively low message overhead during cold discovery in small networks. In the tested recovery scenario, Link State recorded 16 transmission attempts compared with 36 for Distance Vector, a 55.6% reduction in the measured message count. This result reflects the update mechanisms and counting rules implemented in this simulation; it does not establish universal superiority of Link State routing across all networks or failure conditions.
2. **Information Scope:** Distance Vector operates on neighbor advertisements ("routing by rumor"), requiring protective mechanisms such as Split Horizon with Poisoned Reverse to avoid transient counting-to-infinity. Link State provides each router with an identical, complete map of the topology, allowing independent deterministic route calculation via Dijkstra's algorithm.
3. **Model Limitations:** The simulation operates in synchronous discrete rounds on a five-node teaching topology. Production implementations (such as RIP and OSPF) operate asynchronously and incorporate protocol timers, keepalive hellos, designated routers, and area hierarchies, which influence real-world scalability and convergence timing.

6. Simulation Reproduction Guide

To reproduce the simulation and generate the results CSV independently, follow the instructions below:

6.1 Required Files & Dependencies

The simulation relies on two source files that must reside in the same working directory:

- `main.py` : The simulation entry point (contains DV, LS, flooding logic, recovery, and validation).
- `network_config.py` : Required dependency containing `NODES` , `LINKS` , and `LINK_CHANGE` . (`main.py` imports these directly and cannot run without this file).

6.2 Environment Setup & Execution

The code requires Python 3.10+ and the `networkx` library. For generating the graphs, `matplotlib` library is required. Note that `main.py` writes its output to `results/results.csv` , so the `results` subdirectory must be created before execution:

```
# 1. Download main.py and network_config.py into your project directory
# 2. Install the required dependencies
pip install -r requirements.txt

# 3. Create the required output directory (mandatory for CSV output)
mkdir results

# 4. Run the simulation
python main.py
```

6.3 Output Verification

Upon execution, the terminal will print round-by-round progress, routing table dumps, and validation logs ending with:

```
Validation summary:
Baseline | DV | Distances: PASS | Next hops: PASS
Baseline | LS | Distances: PASS | Next hops: PASS
Fresh start after failure | DV | Distances: PASS | Next hops: PASS
Fresh start after failure | LS | Distances: PASS | Next hops: PASS
Recovery after failure | DV | Distances: PASS | Next hops: PASS
Recovery after failure | LS | Distances: PASS | Next hops: PASS

Overall validation: PASS
Results saved to results/results.csv
```

The generated `results/results.csv` will contain the exact empirical metrics recorded in this report.
